



Faculty of Engineering
Analysis And Design Algorithm
Laboratory Manual

NAME:

ENROLLMENT NUMBER:

SUBJECT NAME:

SUBJECT CODE: **BRANCH:**

SEMESTER: **ACADEMIC YEAR:**



Swaminarayan university
Publication

ISBN Number: 978-81-998196-4-1



सर्व विद्या अधिपतये

Certificate

This is to certify that Mr. / Ms. / Mrs.
having enrollment number of Semester ME /
BE / DIPLOMA in has satisfactorily completed his
term working (Subject name) (subject code:)
during academic year at Faculty of Engineering, Swaminarayan University.

Place:

Date:

ISHANI HARSHIL DAVE
SUBJECT IN-CHARGE

ASST.PROF. SHIVANI GAJJAR
HOD

DR. BHAVNA SONI
DEAN

List of Practicals

Sr. No.	Title	Page No.	Date	Sign
1	Implementation and time analysis of sorting algorithms. Bubble sort, selection sort, Insertion sort, Merge sort and Quick sort.			
2	Implementation and Time analysis of linear and binary search algorithm.			
3	Implementation of max-heap sort algorithm.			
4	Implementation and Time analysis of factorial program using iterative and recursive method.			
5	Implementation of a knapsack problem using dynamic programming.			
6	Implementation of chain matrix multiplication using dynamic programming.			
7	Implementation of making a change problem using dynamic programming			
8	Implementation of a knapsack problem using greedy algorithm.			
9	Implementation of Graph and Searching (DFS and BFS).			
10	Implement prim's algorithm.			
11	Implement kruskal's algorithm.			
12	Implement LCS problem.			

Practical-1

AIM: Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort, and Quick sort

```
#include <stdio.h>
#include <stdlib.h>

// Function to print array
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

// ----- Bubble Sort -----
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
}

// ----- Selection Sort -----
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++)
            if (arr[j] < arr[minIdx])
                minIdx = j;

        int temp = arr[minIdx];
        arr[minIdx] = arr[i];
        arr[i] = temp;
    }
}

// ----- Insertion Sort -----
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
```

```
        j--;
    }

    arr[j + 1] = key;
}

// ----- Quick Sort -----
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++)
        if (arr[j] < pivot) {
            i++;
            int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
        }

    int temp = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = temp;
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// ----- Merge Sort -----
void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    int L[n1], R[n2];

    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];

    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
```

```
int m = 1 + (r - 1) / 2;

mergeSort(arr, l, m);
mergeSort(arr, m + 1, r);
merge(arr, l, m, r);
}
}

// ----- Main Program -----
int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);
    int choice;

    printf("Original array: ");
    printArray(arr, n);

    printf("\nChoose Sorting Algorithm:\n");
    printf("1. Bubble Sort\n");
    printf("2. Selection Sort\n");
    printf("3. Insertion Sort\n");
    printf("4. Quick Sort\n");
    printf("5. Merge Sort\n");
    printf("Enter your choice (1-5): ");
    scanf("%d", &choice);

    // Copy original array for sorting
    int copy[100];
    for (int i = 0; i < n; i++)
        copy[i] = arr[i];

    switch (choice) {
        case 1:
            bubbleSort(copy, n);
            printf("Sorted using Bubble Sort: ");
            break;
        case 2:
            selectionSort(copy, n);
            printf("Sorted using Selection Sort: ");
            break;
        case 3:
            insertionSort(copy, n);
            printf("Sorted using Insertion Sort: ");
            break;
        case 4:
            quickSort(copy, 0, n - 1);
            printf("Sorted using Quick Sort: ");
            break;
        case 5:
```

```
        mergeSort(copy, 0, n - 1);
        printf("Sorted using Merge Sort: ");
        break;
default:
    printf("Invalid choice!\n");
    return 1;
}
printArray(copy, n);
return 0;
}
```

Output

Original array: 64 25 12 22 11

Choose Sorting Algorithm:

1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort

Enter your choice (1-5): 4

Sorted using Quick Sort: 11 12 22 25 64

Practical-2

AIM: Implementation and Time analysis of linear and binary search algorithm.

```
#include <stdio.h>
// ----- Linear Search -----
int linearSearch(int arr[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target)
            return i;
    }
    return -1;
}

// ----- Binary Search -----
// NOTE: Binary Search requires sorted array
int binarySearch(int arr[], int n, int target) {
    int low = 0, high = n - 1;

    while (low <= high) {
        int mid = (low + high) / 2;

        if (arr[mid] == target)
            return mid;
        else if (arr[mid] < target)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1;
}

// ----- Utility: Print Array -----
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

// ----- Main Program -----
int main() {
    int arr[] = {3, 8, 15, 20, 24, 32, 45};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target, choice;

    printf("Array: ");
    printArray(arr, n);
```

```
printf("\nEnter element to search: ");
scanf("%d", &target);

printf("\nChoose Search Algorithm:\n");
printf("1. Linear Search\n");
printf("2. Binary Search\n");
printf("Enter your choice (1 or 2): ");
scanf("%d", &choice);

int result = -1;

switch (choice) {
    case 1:
        result = linearSearch(arr, n, target);
        if (result != -1)
            printf("Element found at index %d using Linear Search.\n", result);
        else
            printf("Element not found using Linear Search.\n");
        break;

    case 2:
        result = binarySearch(arr, n, target);
        if (result != -1)
            printf("Element found at index %d using Binary Search.\n", result);
        else
            printf("Element not found using Binary Search.\n");
        break;

    default:
        printf("Invalid choice!\n");
}

return 0;
}
```

Output

Sample Output 1 (Binary Search)

Array: 3 8 15 20 24 32 45

Enter element to search: 24

Choose Search Algorithm:

1. Linear Search

2. Binary Search

Enter your choice (1 or 2): 2

Element found at index 4 using Binary Search.

Sample Output 2 (Linear Search - Not Found)

Array: 3 8 15 20 24 32 45

Enter element to search: 100

Choose Search Algorithm:

1. Linear Search

2. Binary Search

Enter your choice (1 or 2): 1

Element not found using Linear Search.

Practical-3

AIM: Implementation of max-heap sort algorithm.

```
#include <stdio.h>
// Function to swap two integers
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Function to heapify a subtree rooted at index i
void heapify(int arr[], int n, int i) {
    int largest = i;    // Initialize largest as root
    int left = 2 * i + 1;    // Left child
    int right = 2 * i + 2;    // Right child

    // If left child is larger than root
    if (left < n && arr[left] > arr[largest])
        largest = left;

    // If right child is larger than largest so far
    if (right < n && arr[right] > arr[largest])
        largest = right;

    // If largest is not root
    if (largest != i) {
        swap(&arr[i], &arr[largest]);

        // Recursively heapify the affected sub-tree
        heapify(arr, n, largest);
    }
}

// Function to perform heap sort
void heapSort(int arr[], int n) {
    // Build max heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // One by one extract elements
    for (int i = n - 1; i >= 0; i--) {
        // Move current root to end
        swap(&arr[0], &arr[i]);

        // Call max heapify on the reduced heap
        heapify(arr, i, 0);
    }
}
```

```
    }  
}  
  
// Function to print array  
void printArray(int arr[], int n) {  
    for (int i = 0; i < n; ++i)  
        printf("%d ", arr[i]);  
    printf("\n");  
}  
  
// Main function  
int main() {  
    int arr[] = {15, 3, 17, 10, 84, 19, 6, 22, 9};  
    int n = sizeof(arr) / sizeof(arr[0]);  
  
    printf("Original array:\n");  
    printArray(arr, n);  
  
    heapSort(arr, n);  
  
    printf("Sorted array using Max-Heap Sort:\n");  
    printArray(arr, n);  
  
    return 0;  
}
```

Output

Original array:

15 3 17 10 84 19 6 22 9

Sorted array using Max-Heap Sort:

3 6 9 10 15 17 19 22 84

Practical-4

AIM: Implementation and Time analysis of factorial program using iterative and recursive method.

```
#include <stdio.h>

// Function to find factorial using Iterative Method
unsigned long long factorial_iterative(int n)
{
    unsigned long long result = 1;

    for (int i = 2; i <= n; i++)
    {
        result = result * i;
    }

    return result;
}

// Function to find factorial using Recursive Method
unsigned long long factorial_recursive(int n)
{
    if (n == 0 || n == 1)
    {
        return 1;
    }
    else
    {
        return n * factorial_recursive(n - 1);
    }
}

// Main Function
int main()
{
    int num;

    printf("Enter a positive integer: ");
    scanf("%d", &num);

    if (num < 0)
    {
        printf("Factorial is not defined for negative numbers.\n");
    }
    else
    {
        printf("Factorial of %d using Iterative Method = %llu\n",
            num, factorial_iterative(num));
    }
}
```

```
        printf("Factorial of %d using Recursive Method = %llu\n",
               num, factorial_recursive(num));
    }

    return 0;
}
```

Output

Enter a positive integer: 5

Factorial of 5 using Iterative method = 120

Factorial of 5 using Recursive method = 120

Practical-5

AIM: Implementation of a knapsack problem using dynamic programming.

```
#include <stdio.h>

// Function to get the maximum of two integers
int max(int a, int b) {
    return (a > b) ? a : b;
}

// Function to solve the knapsack problem using dynamic programming
int knapsack(int W, int wt[], int val[], int n) {
    int i, w;
    int K[n + 1][W + 1];

    // Build table K[][] in bottom-up manner
    for (i = 0; i <= n; i++) {
        for (w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                K[i][w] = 0;
            else if (wt[i - 1] <= w)
                K[i][w] = max(val[i - 1] + K[i - 1][w - wt[i - 1]], K[i - 1][w]);
            else
                K[i][w] = K[i - 1][w];
        }
    }

    // K[n][W] contains the maximum value that can be put in a knapsack of capacity W
    return K[n][W];
}

// Main function
int main() {
    int val[] = {60, 100, 120}; // Values (profits)
    int wt[] = {10, 20, 30};    // Weights
    int W = 50;                 // Knapsack capacity
    int n = sizeof(val) / sizeof(val[0]);

    int maxProfit = knapsack(W, wt, val, n);

    printf("Maximum value in knapsack of capacity %d = %d\n", W, maxProfit);
    return 0;
}
```

Output

Maximum value in knapsack of capacity 50 = 220

Practical-6

AIM: Implementation of chain matrix multiplication using dynamic programming.

```
#include <stdio.h>
#include <limits.h> // For INT_MAX

// Function to find the minimum number of multiplications
int matrixChainOrder(int p[], int n) {
    int m[n][n]; // m[i][j] = Minimum number of multiplications needed to compute A[i]A[i+1]...A[j] = A[i..j]

    // cost is zero when multiplying one matrix
    for (int i = 1; i < n; i++)
        m[i][i] = 0;

    // l is chain length
    for (int l = 2; l < n; l++) {
        for (int i = 1; i < n - l + 1; i++) {
            int j = i + l - 1;
            m[i][j] = INT_MAX;

            for (int k = i; k <= j - 1; k++) {
                // q = cost/scalar multiplications
                int q = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] * p[j];

                if (q < m[i][j])
                    m[i][j] = q;
            }
        }
    }

    return m[1][n - 1]; // Minimum cost to multiply from matrix 1 to matrix n-1
}

// Main function
int main() {
    // Example: dimensions of matrices A1(10x30), A2(30x5), A3(5x60)
    int arr[] = {10, 30, 5, 60}; // Length = number of matrices + 1
    int n = sizeof(arr) / sizeof(arr[0]);

    int minCost = matrixChainOrder(arr, n);

    printf("Minimum number of scalar multiplications is %d\n", minCost);

    return 0;
}
```

Output

Minimum number of scalar multiplications is 4500

Practical-7

AIM: Implementation of making a change problem using dynamic programming.

```
#include <stdio.h>
#include <limits.h> // For INT_MAX

// Function to find the minimum number of coins
int minCoins(int coins[], int n, int amount) {
    int dp[amount + 1];

    // Initialize all dp values as infinite
    for (int i = 0; i <= amount; i++)
        dp[i] = INT_MAX;

    dp[0] = 0; // Base case: 0 coins are needed to make amount 0

    // Compute minimum coins required for all values from 1 to amount
    for (int i = 1; i <= amount; i++) {
        for (int j = 0; j < n; j++) {
            if (coins[j] <= i && dp[i - coins[j]] != INT_MAX) {
                if (dp[i] > dp[i - coins[j]] + 1)
                    dp[i] = dp[i - coins[j]] + 1;
            }
        }
    }

    return (dp[amount] == INT_MAX) ? -1 : dp[amount];
}

// Main function
int main() {
    int coins[] = {1, 3, 4}; // Available coin denominations
    int n = sizeof(coins) / sizeof(coins[0]);
    int amount = 6;

    int result = minCoins(coins, n, amount);

    if (result != -1)
        printf("Minimum coins required to make %d = %d\n", amount, result);
    else
        printf("It is not possible to make %d with given coins.\n", amount);

    return 0;
}
```

Output

Minimum coins required to make 6 = 2

Practical-8

AIM: Implementation of a knapsack problem using greedy algorithm

```
#include <stdio.h>
#include <stdlib.h>

// Structure to store weight, value, and value/weight ratio
struct Item {
    int weight, value;
    float ratio;
};

// Compare function to sort items by ratio (descending)
int compare(const void *a, const void *b) {
    struct Item *item1 = (struct Item *)a;
    struct Item *item2 = (struct Item *)b;
    if (item1->ratio < item2->ratio) return 1;
    else if (item1->ratio > item2->ratio) return -1;
    else return 0;
}

// Function to solve Fractional Knapsack
float fractionalKnapsack(struct Item items[], int n, int capacity) {
    // Calculate value/weight ratio
    for (int i = 0; i < n; i++)
        items[i].ratio = (float)items[i].value / items[i].weight;

    // Sort items by ratio in descending order
    qsort(items, n, sizeof(struct Item), compare);

    float totalValue = 0.0;
    int currentWeight = 0;

    for (int i = 0; i < n; i++) {
        if (currentWeight + items[i].weight <= capacity) {
            // Take whole item
            currentWeight += items[i].weight;
            totalValue += items[i].value;
        } else {
            // Take fractional part of item
            int remain = capacity - currentWeight;
            totalValue += items[i].ratio * remain;
            break;
        }
    }

    return totalValue;
}
```

```
}

// Main function
int main() {
    struct Item items[] = {
        {10, 60},
        {20, 100},
        {30, 120}
    };
    int n = sizeof(items) / sizeof(items[0]);
    int capacity = 50;

    float maxValue = fractionalKnapsack(items, n, capacity);

    printf("Maximum value in knapsack of capacity %d = %.2f\n", capacity, maxValue);

    return 0;
}
```

Output

Maximum value in knapsack of capacity 50 = 240.00

Practical-9

AIM: Implementation of Graph and Searching (DFS and BFS).

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

// Structure for adjacency list node
struct Node {
    int vertex;
    struct Node* next;
};

// Structure for graph
struct Graph {
    int numVertices;
    struct Node** adjLists;
    int* visited;
};

// Queue structure for BFS
struct Queue {
    int items[MAX];
    int front, rear;
};

// Function to create a node
struct Node* createNode(int v) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->vertex = v;
    newNode->next = NULL;
    return newNode;
}

// Create a graph
struct Graph* createGraph(int vertices) {
    struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
    graph->numVertices = vertices;

    graph->adjLists = (struct Node**)malloc(vertices * sizeof(struct Node*));
    graph->visited = (int*)malloc(vertices * sizeof(int));

    for (int i = 0; i < vertices; i++) {
        graph->adjLists[i] = NULL;
        graph->visited[i] = 0;
    }
}
```

```
    return graph;
}

// Add edge (undirected)
void addEdge(struct Graph* graph, int src, int dest) {
    // Add edge from src to dest
    struct Node* newNode = createNode(dest);
    newNode->next = graph->adjLists[src];
    graph->adjLists[src] = newNode;

    // Since it's undirected, add edge from dest to src
    newNode = createNode(src);
    newNode->next = graph->adjLists[dest];
    graph->adjLists[dest] = newNode;
}

// DFS function
void DFS(struct Graph* graph, int vertex) {
    graph->visited[vertex] = 1;
    printf("%d ", vertex);

    struct Node* temp = graph->adjLists[vertex];
    while (temp) {
        int connectedVertex = temp->vertex;
        if (!graph->visited[connectedVertex])
            DFS(graph, connectedVertex);
        temp = temp->next;
    }
}

// Queue functions for BFS
struct Queue* createQueue() {
    struct Queue* q = (struct Queue*)malloc(sizeof(struct Queue));
    q->front = -1;
    q->rear = -1;
    return q;
}

int isEmpty(struct Queue* q) {
    return q->rear == -1;
}

void enqueue(struct Queue* q, int value) {
    if (q->rear == MAX - 1)
        return;
    if (q->front == -1)
        q->front = 0;
    q->items[++q->rear] = value;
}
```

```
}
```

```
int dequeue(struct Queue* q) {  
    if (isEmpty(q))  
        return -1;  
    int item = q->items[q->front++];  
    if (q->front > q->rear)  
        q->front = q->rear = -1;  
    return item;  
}
```

```
// BFS function
```

```
void BFS(struct Graph* graph, int startVertex) {  
    struct Queue* q = createQueue();  
  
    graph->visited[startVertex] = 1;  
    enqueue(q, startVertex);  
  
    while (!isEmpty(q)) {  
        int currentVertex = dequeue(q);  
        printf("%d ", currentVertex);  
  
        struct Node* temp = graph->adjLists[currentVertex];  
        while (temp) {  
            int adjVertex = temp->vertex;  
            if (!graph->visited[adjVertex]) {  
                graph->visited[adjVertex] = 1;  
                enqueue(q, adjVertex);  
            }  
            temp = temp->next;  
        }  
    }  
}
```

```
// Function to reset visited array
```

```
void resetVisited(struct Graph* graph) {  
    for (int i = 0; i < graph->numVertices; i++)  
        graph->visited[i] = 0;  
}
```

```
// Main function
```

```
int main() {  
    int vertices = 6;  
  
    struct Graph* graph = createGraph(vertices);  
  
    addEdge(graph, 0, 1);  
    addEdge(graph, 0, 2);  
    addEdge(graph, 1, 3);  
}
```

```
addEdge(graph, 1, 4);
addEdge(graph, 2, 4);
addEdge(graph, 3, 5);
addEdge(graph, 4, 5);

printf("DFS traversal starting from vertex 0: ");
DFS(graph, 0);

resetVisited(graph); // Reset before BFS
printf("\nBFS traversal starting from vertex 0: ");
BFS(graph, 0);

printf("\n");

return 0;
}
```

Output

DFS traversal starting from vertex 0: 0 2 4 5 1 3
BFS traversal starting from vertex 0: 0 2 1 4 5 3

Practical-10

AIM: Implement prim's algorithm

```
#include <stdio.h>
#include <limits.h>

#define MAX 100
#define INF 9999

int minKey(int key[], int mstSet[], int V) {
    int min = INF, min_index;

    for (int v = 0; v < V; v++)
        if (mstSet[v] == 0 && key[v] < min)
            min = key[v], min_index = v;

    return min_index;
}

void printMST(int parent[], int graph[MAX][MAX], int V) {
    int totalCost = 0;
    printf("Edge \tWeight\n");
    for (int i = 1; i < V; i++) {
        printf("%d - %d \t%d \n", parent[i], i, graph[i][parent[i]]);
        totalCost += graph[i][parent[i]];
    }
    printf("Total cost of MST: %d\n", totalCost);
}

void primMST(int graph[MAX][MAX], int V) {
    int parent[MAX]; // Array to store constructed MST
    int key[MAX];    // Key values used to pick minimum weight edge
    int mstSet[MAX]; // To represent set of vertices included in MST

    // Initialize all keys as INFINITE
    for (int i = 0; i < V; i++)
        key[i] = INF, mstSet[i] = 0;

    // Start from the first vertex
    key[0] = 0; // Make key 0 so that this vertex is picked first
    parent[0] = -1; // First node is always root of MST

    for (int count = 0; count < V - 1; count++) {
        // Pick the minimum key vertex not yet included in MST
        int u = minKey(key, mstSet, V);

        mstSet[u] = 1; // Add the picked vertex to the MST Set
```

```
// Update key and parent index of the adjacent vertices
for (int v = 0; v < V; v++)
    if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v])
        parent[v] = u, key[v] = graph[u][v];
}

// Print the constructed MST
printMST(parent, graph, V);
}

// Main function
int main() {
    int V = 5; // Number of vertices

    // Example adjacency matrix
    int graph[MAX][MAX] = {
        {0, 2, 0, 6, 0},
        {2, 0, 3, 8, 5},
        {0, 3, 0, 0, 7},
        {6, 8, 0, 0, 9},
        {0, 5, 7, 9, 0},
    };

    printf("Prim's Minimum Spanning Tree:\n");
    primMST(graph, V);

    return 0;
}
```

Output

Prim's Minimum Spanning Tree:

Edge	Weight
------	--------

0 - 1	2
-------	---

1 - 2	3
-------	---

0 - 3	6
-------	---

1 - 4	5
-------	---

Total cost of MST: 16

Practical-11

AIM: Implement kruskal's algorithm.

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

// Structure to represent an edge
struct Edge {
    int src, dest, weight;
};

// Structure to represent a graph
struct Graph {
    int V, E;
    struct Edge edges[MAX];
};

// Find parent using Union-Find with path compression
int find(int parent[], int i) {
    if (parent[i] != i)
        parent[i] = find(parent, parent[i]);
    return parent[i];
}

// Union of two subsets
void unionSet(int parent[], int rank[], int x, int y) {
    int xroot = find(parent, x);
    int yroot = find(parent, y);

    if (rank[xroot] < rank[yroot])
        parent[xroot] = yroot;
    else if (rank[xroot] > rank[yroot])
        parent[yroot] = xroot;
    else {
        parent[yroot] = xroot;
        rank[xroot]++;
    }
}

// Compare edges by weight (used in qsort)
int compareEdges(const void* a, const void* b) {
    struct Edge* e1 = (struct Edge*)a;
    struct Edge* e2 = (struct Edge*)b;
    return e1->weight - e2->weight;
}
```

```
// Function to run Kruskal's algorithm
void kruskalMST(struct Graph* graph) {
    int V = graph->V;
    struct Edge result[MAX]; // Store result edges
    int e = 0; // Index for result[]
    int i = 0; // Index for sorted edges

    // Step 1: Sort all edges in non-decreasing order
    qsort(graph->edges, graph->E, sizeof(graph->edges[0]), compareEdges);

    // Allocate memory for Union-Find
    int parent[V];
    int rank[V];

    // Create subsets with single elements
    for (int v = 0; v < V; v++) {
        parent[v] = v;
        rank[v] = 0;
    }

    // Number of edges to be taken is V-1
    while (e < V - 1 && i < graph->E) {
        struct Edge next = graph->edges[i++];

        int x = find(parent, next.src);
        int y = find(parent, next.dest);

        // If including this edge doesn't cause cycle
        if (x != y) {
            result[e++] = next;
            unionSet(parent, rank, x, y);
        }
    }

    // Print the result
    printf("Edges in the Minimum Spanning Tree:\n");
    int totalWeight = 0;
    for (i = 0; i < e; i++) {
        printf("%d - %d : %d\n", result[i].src, result[i].dest, result[i].weight);
        totalWeight += result[i].weight;
    }
    printf("Total cost of MST: %d\n", totalWeight);
}

// Main function
int main() {
    struct Graph graph;
    graph.V = 4; // Number of vertices
```

```
graph.E = 5; // Number of edges

// Define edges: src, dest, weight
graph.edges[0] = (struct Edge){0, 1, 10};
graph.edges[1] = (struct Edge){0, 2, 6};
graph.edges[2] = (struct Edge){0, 3, 5};
graph.edges[3] = (struct Edge){1, 3, 15};
graph.edges[4] = (struct Edge){2, 3, 4};

printf("Kruskal's Minimum Spanning Tree:\n");
kruskalMST(&graph);

return 0;
}
```

Output

Kruskal's Minimum Spanning Tree:

Edges in the Minimum Spanning Tree:

2 - 3 : 4

0 - 3 : 5

0 - 1 : 10

Total cost of MST: 19

Practical-12

AIM: Implement LCS problem

```
#include <stdio.h>
#include <string.h>

// Function to find LCS length and print LCS string
int lcs(char *X, char *Y, int m, int n) {
    int L[m+1][n+1];

    // Build the LCS table in bottom-up fashion
    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n; j++) {
            if (i == 0 || j == 0)
                L[i][j] = 0;
            else if (X[i-1] == Y[j-1])
                L[i][j] = L[i-1][j-1] + 1;
            else
                L[i][j] = (L[i-1][j] > L[i][j-1]) ? L[i-1][j] : L[i][j-1];
        }
    }

    // Length of LCS is L[m][n]
    int index = L[m][n];

    // Create a character array to store the LCS string
    char lcsStr[index + 1];
    lcsStr[index] = '\0'; // Null terminate

    // Start from bottom right corner and find the LCS string
    int i = m, j = n;
    while (i > 0 && j > 0) {
        // If current characters are same, it's part of LCS
        if (X[i-1] == Y[j-1]) {
            lcsStr[index - 1] = X[i - 1];
            i--;
            j--;
            index--;
        }
        // Else move in the direction of the larger value
        else if (L[i-1][j] > L[i][j-1])
            i--;
        else
            j--;
    }

    printf("Longest Common Subsequence: %s\n", lcsStr);
```

```
    return L[m][n];
}

int main() {
    char X[] = "AGGTAB";
    char Y[] = "GXTXAYB";
    int m = strlen(X);
    int n = strlen(Y);
    int length = lcs(X, Y, m, n);
    printf("Length of LCS: %d\n", length);

    return 0;
}
```

Output

Longest Common Subsequence: GTAB

Length of LCS: 4